
CSE 151B/251B Milestone Report: Inference-Time Engineering for Math Reasoning with a Quantized Thinking LLM

Connor Wu

Department of Computer Science and Engineering
University of California, San Diego
wu.connor22@gmail.com

Abstract

The CSE 151B Spring 2026 competition asks us to maximise the accuracy of a fixed, INT8-quantised Qwen3-4B-Thinking-2507 model on a benchmark of 1126 mixed multiple-choice (MCQ) and free-form math problems. With no training permitted by our current setup, all gains must come from inference-time engineering: prompt construction, decoding parameters, answer extraction, and input normalisation. This milestone documents a fast iteration loop in which a heavily “optimised” notebook initially performed *worse* than the starter (3/10 overall on a development slice versus the starter’s 3/5 on a 5-question smoke test). We trace the regression to five interacting bugs (truncated thinking budget, a no-think calibration pass that overrode the thinking response, an unscoped LaTeX repair that corrupted English text, `enforce_eager` disabling CUDA graphs, and lower `top_p`) and fix them with low-runtime-cost changes. **Final private Kaggle score: TODO – to be added once the full submission is graded.**

1 Introduction

Problem Definition. We are given a math-reasoning benchmark of $n=1126$ items (public split) in which each item is either (i) a multiple-choice question (MCQ; 375 items) with up to ten labelled options $A, B, \dots, J$, or (ii) a free-form problem (751 items) whose ground-truth answer is one or more symbolic or numeric expressions. Inputs are a question string q (in informal LaTeX) plus an optional option list $O = (o_1, \dots, o_k)$; the prediction is a single letter (MCQ) or one or more expressions (free-form). A symbolic judge checks free-form answers for mathematical equivalence rather than string match. The generative model is fixed at Qwen3-4B-Thinking-2507, served by vLLM in INT8 BitsAndBytes quantisation; no fine-tuning or external solver call is part of the current pipeline.

Problem Significance. Quantised inference of mid-sized open-weight reasoning models is the deployment setting most students, hobbyists, and small companies actually face: training budgets are out of reach, but a single consumer GPU can hold a 4–7B model in 8-bit. Understanding which inference-time choices unlock or destroy accuracy in this regime, especially for “thinking” models that produce a long `<think>...</think>` chain before the final answer, is directly transferable to real applications where compute is constrained and post-hoc improvements are the only knobs available.

Technical Challenge. Four challenges shape this project. (1) *Data quality*: the dataset stores LaTeX with the backslashes stripped (`frac`, `infy`, `int`, etc.), so the model must either parse degraded mathematics or the strings must be repaired before prompting – but a naive repair corrupts common English words (“sum”, “times”, “log”, “pi”) that share spellings with LaTeX commands.

(2) *Thinking-model output handling*: the model emits a long internal chain before a final boxed answer; a too-tight token budget truncates before the answer ever appears. (3) *Multi-faceted scoring*: MCQ scoring is straightforward letter match, but free-form scoring uses a symbolic judger that demands exactly one well-formed `\boxed{\}`, so prompt design matters. (4) *Honest ablation*: small changes interact, so a multi-knob “optimised” notebook can regress without any single knob looking wrong in isolation; we needed a disciplined diff against the starter to localise the regression.

Contributions. The starter code provides a working end-to-end pipeline: vLLM inference with a generous 32,768-token budget, simple MCQ/free-form system prompts, a regex letter extractor, and the course-provided symbolic judger. Its limitations are that no LaTeX repair is performed, MCQ extraction never abstains (it falls back to the last capital letter found anywhere in the response), nested-brace `\boxed{\}` contents are not parsed, and decoding uses a single fixed sampling setting with no calibration option. An initial attempt at “optimising” the starter regressed substantially in a development run (3/10 versus the starter’s 3/5). Our milestone contributions are:

- **A diagnosis of five interacting regressions** (Section 3) that explains why an apparently well-motivated optimisation pass lost accuracy on a thinking model. The diagnosis is the main technical content of this milestone.
- **Scoped LaTeX repair** that restores missing backslashes only inside dollar-delimited math, preserving plain English while still fixing corrupted formulae.
- **A lazy-fallback MCQ pipeline** that uses the thinking response as the primary answer and only invokes a no-think first-token logprobs pass on the small fraction of items where extraction abstains, eliminating the previous mass-override of strong answers by weak ones.
- **Hardened answer extraction** with a brace-counting `extract_boxed` and a `</think>`-aware `extract_letter` that preferentially reads from the post-thinking region.
- **An open, reproducible patch** delivered as a single diff against `optimized_notebook.ipynb` with the original retained as `.bak`; the change is runtime-flat or slightly faster than the broken version it replaces.

2 Related Work

Reasoning LLMs. Recent open-weight “thinking” models such as the Qwen3 series [3] interleave a hidden chain of thought, delimited by `<think></think>`, with a final user-facing answer. Their accuracy on math benchmarks improves with adequate decoding budget and suitable sampling temperature; recommended defaults are documented by the model authors.

Inference serving. vLLM [2] provides batched, paged attention serving of HuggingFace models, including BitsAndBytes 8-bit weight quantisation. Decoding throughput depends on `max_num_seqs`, `enable_prefix_caching`, and whether CUDA graphs are enabled (`enforce_eager=False`).

Symbolic answer judging. Math benchmarks such as MATH [1] have driven the design of symbolic equivalence checkers built on SymPy-like CAS backends; the course-provided `Judger` class follows this lineage and accepts free-form responses, extracting and normalising the boxed expression internally.

3 Methods

Problem Setting. Let $\mathcal{D} = \{(q_i, O_i, y_i)\}_{i=1}^n$ be the public set with O_i possibly empty. The model f_θ is held fixed; we choose a prompt template π , sampling parameters σ , and extraction function g to maximise

$$\text{Acc}(\pi, \sigma, g) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[\text{Judge}(g(f_\theta(\pi(q_i, O_i); \sigma))) = y_i],$$

with `Judge` being letter equality for MCQ and symbolic equivalence for free-form. There is no trainable loss in this milestone; all optimisation is over the discrete choices (π, σ, g) .

Idea Summary. The five-bug diagnosis is summarised in Table 1. Each fix is a small, localised change, and several of them *reduce* compute (lazy calibration; removing `enforce_eager`). The intent is to recover what the optimised notebook should have been: a strict superset of the starter that exploits LaTeX repair and a fallback MCQ extractor without overriding the thinking-model’s strong primary answer.

Table 1: Five interacting regressions identified in the original `optimized_notebook.ipynb` and their fixes. “Impact” is the author’s subjective ranking based on the development run and on first principles.

#	Regression in v1	Fix in v2	Impact
1	<code>MAX_TOKENS=4096</code> truncates the thinking chain before <code>\boxed{}</code> ever appears.	Restore to 16,384 (starter used 32,768); typical responses finish well below this so runtime impact is modest.	High
2	The MCQ “logprobs calibration” pass ran for every item and overrode the thinking response with a no-think first-token argmax.	Use <code>extract_letter</code> on the thinking response as the primary prediction; invoke calibration only when extraction abstains.	High
3	LaTeX repair rewrote bare words <code>sum</code> , <code>times</code> , <code>log</code> , <code>pi</code> , <code>delta</code> , etc., even in plain-English problem statements (e.g. “Find the sum of...” → “Find the <code>\sum</code> of...”).	Scope all substitutions to text inside <code>\$...\$</code> math delimiters; English left untouched.	Medium
4	<code>enforce_eager=True</code> disabled CUDA graphs without any accuracy justification.	Remove; let vLLM compile.	Low (speed)
5	Sampling drifted from the Qwen3 recommendation (<code>temperature=0.6</code> , <code>top_p=0.95</code>) to (0.7, 0.8), and several brittle <code>stop</code> strings could fire mid-reasoning.	Revert to recommended sampling; drop <code>stop=[...]</code> .	Low–Medium

In addition to those, we hardened answer extraction in two small ways: `extract_letter` now reads preferentially from the substring *after* the closing `</think>` tag (and only falls back to the full response if nothing is found), and `extract_boxed` uses a brace-counting parser so that nested arguments such as `\boxed{\frac{1}{2}}` are extracted in full rather than being truncated at the first inner closing brace.

4 Experiments

4.1 Baselines

We compare three configurations on the public development data:

- **Starter** – the unmodified course-provided notebook. Single vLLM pass with `MAX_TOKENS=32768`, no LaTeX repair, no MCQ calibration, regex letter extraction that never abstains.
- **Optimized v1 (regressed)** – our first attempt: `MAX_TOKENS=4096`, global LaTeX repair (not scoped to math), two-pass MCQ scoring where a no-think calibration pass overrides the thinking response, `enforce_eager=True`, and `temperature/top_p` of 0.7/0.8.
- **Optimized v2 (this milestone)** – the fix described in Section 3: 16,384-token budget, scoped LaTeX repair, lazy-fallback MCQ pipeline, recommended Qwen3 sampling, hardened answer extraction.

4.2 Evaluation

For each question we record whether the predicted answer is correct under the appropriate judger. MCQ predictions are scored by case-insensitive letter equality with the gold option label. Free-form predictions are scored by the course-provided symbolic Judger, which extracts the boxed expression from the response and tests mathematical equivalence against each gold answer. Aggregate accuracies are reported separately for MCQ and free-form, plus an overall figure.

4.3 Implementation details

Compute. All experiments use a single CUDA device. The model is Qwen/Qwen3-4B-Thinking-2507 (~4B parameters), loaded into vLLM 0.20.2 with `quantization="bitsandbytes"`, `load_format="bitsandbytes"`, `max_model_len=16384`, and `gpu_memory_utilization=0.50`. Reported GPU footprint at load is ≈ 2.7 GiB plus an 8.3 GiB KV cache, fitting comfortably in ~16 GiB of usable VRAM.

Sampling. v1 used the Qwen-chat “creative” setting ($T=0.7$, $p=0.8$); v2 reverts to the Qwen3-Thinking recommended setting ($T=0.6$, $p=0.95$, `top_k=20`) with `repetition_penalty=1.0`. v2 sets `max_tokens=16,384` and removes the brittle `stop=["</answer>", "\n\nProblem", "\n\nQuestion"]` list, since `</answer>` is never emitted by this model and the others can match mid-reasoning. The fallback MCQ logprobs pass uses near-greedy $T=0.01$, $p=1.0$, and asks for `logprobs=20`.

Prompting. Both system prompts are tuned to the thinking model. The MCQ prompt explicitly instructs “After `</think>`, output exactly one line: `\boxed{X}`.” The free-form prompt includes two short in-context examples that demonstrate the single-boxed format with comma-separated sub-answers. LaTeX repair is applied to the question text and (for MCQ) to each option before assembly.

Reproducibility. The patch is applied by a single Python script (`patch_notebook.py`) that rewrites cells 5, 9, 13, 15, and 17 of `optimized_notebook.ipynb`; the pre-patch state is preserved at `optimized_notebook.ipynb.bak`.

Add public GitHub URL for final report.

4.4 Results

Preliminary, $N=10$ development slice. Table 2 reports accuracy on the first ten questions of the public set, used as a fast smoke test during the diagnosis pass. The starter column shows the $N=5$ printout produced by the unmodified notebook (the starter cell only runs five items by default); the two “Optimized” columns are evaluated on the first ten items. *These slices are too small to distinguish v2 from the starter statistically* – their value is in confirming that v1’s MCQ collapse to 0/3 is recovered. A full-dataset run on the patched notebook is in progress and will replace these numbers in the final report.

Table 2: Preliminary accuracy on development slices of the public set. Starter is evaluated on the first 5 items because that is the default in the provided notebook; Optimized v1 and v2 are evaluated on the first 10. v1’s MCQ 0/3 is the smoking gun for the no-think calibration override discussed in Table 1.

Setting	MCQ (correct/total)	Free-form (correct/total)	Overall
Starter ($N=5$)	1/2 (50%)	2/3 (66.7%)	3/5 (60.0%)
Optimized v1 ($N=10$)	0/3 (0.0%)	3/7 (42.9%)	3/10 (30.0%)
Optimized v2 ($N=10$)	full-dataset evaluation pending; expected to recover and exceed starter		

Qualitative observations. Inspecting v1’s responses confirms the diagnosis. The MCQ “calibration” prompt – which disables thinking and asks the model to emit a single letter – collapses to a low-entropy first-token argmax that is wrong on every sampled question; the corresponding thinking response, by contrast, contains a plausible `\boxed{letter}` answer in two of the three cases but is discarded by the scoring path. Similarly, the unscoped LaTeX repair visibly mangles plain-English

problem statements (“Find the \sum of the first \$325\$...”) – a transformation unambiguously visible in v1’s printed prompt-preview cell and absent in v2.

Runtime. v2’s wall-clock cost is comparable to, and likely slightly below, v1’s. The larger `max_tokens` ceiling increases the worst-case per-request budget but most responses terminate on EOS well before 16,384 tokens; removing `enforce_eager` reinstates CUDA graphs in vLLM; and the calibration pass that previously ran on *every* MCQ now runs only on items where extraction abstains – a small minority in practice.

5 Discussion

This milestone replaced a buggy “optimisation” pass with a corrected version that retains the genuinely useful ideas (LaTeX repair, hardened answer extraction, the existence of a calibration fallback) while removing the interactions that destroyed accuracy. The core insight is that *thinking models are exquisitely sensitive to anything that prevents the final answer from surfacing*: too small a token budget, an override path that prefers the no-think pass, and a `stop=[...]` list that can fire during reasoning all act as silent answer-suppressors.

Achievements. We delivered (i) a five-point diagnosis of a real accuracy regression, (ii) a runtime-flat patch that recovers the strong baseline behaviour on the $N=10$ development slice, (iii) an extraction layer that abstains rather than guesses on MCQ, and (iv) a reproducible scripted patch that leaves the original notebook intact as a baseline.

Bottlenecks and Mitigation. The dominant bottleneck on a single GPU is decoding latency for the thinking chain. Running the full 1126-question evaluation once already takes a non-trivial fraction of an hour on our hardware, which makes hyperparameter sweeps expensive and was the main reason the v1 regression was not caught sooner: development was being driven by a 5- to 10-question smoke test. We now plan to (a) maintain a frozen ~ 100 -item stratified subsample (75 free-form, 25 MCQ) as a stable evaluation surface, and (b) before any multi-knob change, baseline both versions on that subset to localise regressions.

Plans Forward. Three directions are queued: (i) majority voting / self-consistency over $n=5$ samples on free-form, scored by the symbolic judger; (ii) a no-think first-pass that produces a cheap candidate answer used only to *filter* which items go through the expensive thinking pass on rerun; and (iii) a small post-processor that normalises the boxed expression (e.g. collapses $325*(1+325)$ to a numeric form when both sides are exactly representable) to make the judger’s job easier on borderline cases. None of these require model training and all keep the model frozen.

Team Responsibilities. This is a solo submission. Connor Wu is responsible for the full pipeline, diagnosis, fixes, evaluation harness, and report.

Timelines.

- Weeks 1–3: Set up environment, run starter notebook end-to-end, read the dataset format and judger internals. (*Complete.*)
- Weeks 4–5: First optimisation pass; diagnosed five-bug regression; produced v2 patch; wrote this milestone. (*Complete.*)
- Week 6: Full $N=1126$ evaluation of v2; record final dev metrics and a first leaderboard submission.
- Week 7: Self-consistency / majority voting for free-form, with a compute budget cap.
- Week 8: Two-pass filtering (cheap candidate $\rightarrow$ thinking rerun on uncertain items only).
- Week 9: Post-processing pipeline (numeric normalisation; targeted retries on judger-soft cases).
- Week 10: Final ablations, frozen evaluation, and submission.

Adjust this timeline
once final-quarter
dates are confirmed.

LLM Usage

LLMs were used in this project, and per the course policy this section describes their role precisely.

Tool used. Anthropic’s Claude (specifically the Claude desktop “Cowork” interface, model `claude-opus-4-7`) was used as an interactive coding assistant with direct read/write access to this repository.

What Claude did. Claude was given the starter notebook and the v1 “optimised” notebook and asked to identify why v1 had regressed in accuracy. It performed the diff, produced the five-bug diagnosis summarised in Table 1, and wrote a Python patch script (`patch_notebook.py`) that surgically rewrote cells 5, 9, 13, 15, and 17 of `optimized_notebook.ipynb`. After running the patch and confirming the notebook remained valid JSON, Claude also drafted the first version of this report based on a short briefing from the author about scope, authorship, and which numbers to cite.

What Claude did not do. Claude did not run experiments on real hardware (the author ran the patched notebook), did not invent results, and did not author this report unsupervised: every claim, number, and diagnosis here was reviewed and accepted (or revised) by the author. The author takes full responsibility for the contents.

Why this matters. Most of the value Claude added in this project was as a fast pair-programmer for debugging an inference pipeline whose failure mode was subtle (a multi-knob regression with no single “wrong” knob). The five-bug diagnosis is the kind of work that is much faster collaboratively with an LLM coding assistant than alone, but the work to verify it (re-running the notebook, eyeballing the prompt-preview outputs, confirming the JSON-patched cells executed) remained with the author.

References

- [1] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [3] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.